

Java TreeSet

The Power of Self-Sorting Data Structures

CODEHIVE

CODEHIVE

1. What is a TreeSet?

While a HashSet is great for uniqueness, it doesn't care about the order of its elements. If you need a collection that is **always unique** and **always sorted**, you use a TreeSet.

The TreeSet class in Java implements the NavigableSet interface, which in turn extends SortedSet. It organizes elements in their natural ascending order (A-Z for Strings, 1-10 for Numbers).

Key Rule: TreeSet does NOT allow null elements. Because it needs to compare every new element with existing ones, a NullPointerException will occur if you try to add a null value.

2. Implementation Example

Let's look at how the `TreeSet` automatically handles sorting and duplicate prevention simultaneously.

```
import java.util.TreeSet; public class Main { public static void
main(String[] args) { TreeSet<String> cars = new TreeSet<>();
cars.add("Volvo"); cars.add("BMW"); cars.add("Ford"); cars.add("BMW"); //
This duplicate is ignored cars.add("Mazda"); // Elements will print in
Alphabetical Order: // BMW, Ford, Mazda, Volvo for(String car : cars)
{ System.out.println(car); } } }
```

3. Comparison: Hash vs. Tree

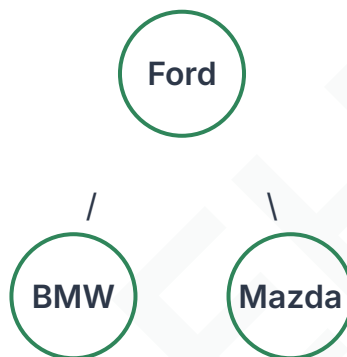
Choosing between these two depends on whether you prioritize **speed** or **order**.

Feature	HashSet	TreeSet
Sorting	None / Random	Natural Order
Search Speed	$O(1)$ - Constant	$O(\log n)$ - Logarithmic
Null Allowed?	Yes (One)	No (Throws Exception)

4. Under the Hood: Red-Black Trees

TreeSet doesn't use an array or buckets. It uses a **Red-Black Tree**, which is a type of self-balancing binary search tree.

Every time you add an element, the tree re-arranges itself to ensure that the "left" child is always smaller and the "right" child is always larger. This structure is what allows it to stay sorted without you ever calling a `sort()` method.



5. Time Complexity: $O(\log n)$

Because it's a tree, the search time is **logarithmic**. What does that mean?

- In an ArrayList of 1,000 items, you might check 1,000 times to find something.
- In a TreeSet of 1,000 items, you only need about **10 comparisons**.
- In a TreeSet of 1,000,000 items, you only need about **20 comparisons**.

While slower than a HashSet, it is still incredibly efficient for huge datasets that need to stay organized.

6. Navigation Methods

Because it implements `NavigableSet`, you get powerful methods that other sets don't have.

- `first()`: Returns the smallest element.
- `last()`: Returns the largest element.
- `higher(val)`: Smallest element strictly greater than `val`.
- `lower(val)`: Largest element strictly less than `val`.
- `descendingSet()`: Flips the order of the set.

```
System.out.println(cars.first()); // BMW
System.out.println(cars.higher("Mazda")); // Volvo
```

7. Sorted Subset Operations

TreeSet allows you to "slice" the set effectively. This is perfect for range-based searches.

```
// Get all cars from "B" up to "M" SortedSet<String> sub = cars.subSet("BMW",  
"Mazda"); // Get everything before "Ford" SortedSet<String> head =  
cars.headSet("Ford");
```

The resulting subsets are **backed** by the original set. If you add a car to the main set that fits in the range, it automatically shows up in the subset!

8. Real World Case: High-Score Leaderboard

Imagine a game where you need to keep the top 10 unique scores visible at all times.

```
TreeSet<Integer> scores = new TreeSet<>(); scores.add(1500);  
scores.add(2300); scores.add(900); // Print Top Score  
System.out.println("Leader: " + scores.last()); // Get players near a  
specific score for matchmaking Integer nearestNeighbor =  
scores.floor(2000); // 1500
```

CODEF

9. Final Verdict

Use TreeSet when:

- You need unique elements that are always sorted.
- You need to perform range-based queries (e.g., "Find all IDs between 500 and 1000").
- Natural ordering is required for the output.

© 2026 CodeHive Academy. This concludes the Java Collections Masterclass.

Next Chapter: Java HashMap Mastery